SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

   Ans:

   Explanation

   A smartphone may experience lag when switching between applications because the processor frequently needs to access data from memory. The memory system is organized as a hierarchy to reduce the time required to access frequently used data.

   L1 cache is the smallest and fastest cache and is located very close to the CPU core. It stores the most frequently accessed instructions and data. **L2 cache** is larger than L1 but slightly slower, while **L3 cache** is larger still and is generally shared among processor cores. **DRAM** provides much larger storage capacity than the caches but has considerably higher access latency.

   To reduce application-switching latency, the smartphone should use a hierarchical memory design in which frequently accessed application data is kept in L1 and L2 caches, shared data is placed in L3 cache, and larger application data is stored in DRAM. The use of larger and faster caches, efficient cache replacement, and good prefetching can reduce the number of slow DRAM accesses.

This redesign reduces memory access latency and improves processor throughput because the CPU spends less time waiting for data.

Proposed Hierarchy

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

- L1: Very fast, very small
- L2: Fast, medium-sized
- L3: Larger shared cache
- DRAM: Large capacity, higher latency

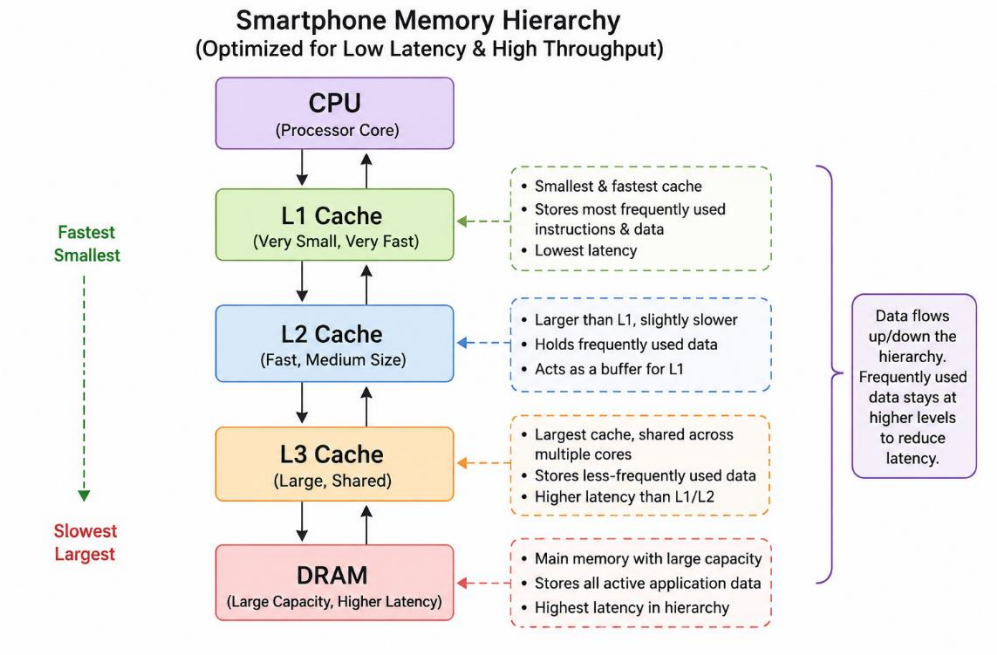
Output:

```
Online C Compiler

Smartphone Memory Simulation
L1/L2/L3 cache -> fast access
DRAM -> larger but slower storage
Data processed: 100000
Sum = 4999950000
Execution time = 0.000000 seconds

Program executed successfully.
```

Diagram:



C Program

The program simulates repeated data access and demonstrates the concept of fast cache access and larger but slower DRAM storage.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

   Ans:

   Explanation

   Cloud servers running databases can experience high query latency when the processor cannot find frequently requested data in the cache. A **cache hierarchy** keeps frequently accessed database information close to the CPU, reducing access time.

   Cache memory is much faster than DRAM. When the required data is present in the cache, a **cache hit** occurs and the processor can access it quickly. When the data is absent, a **cache miss** occurs and the processor must access lower-level memory.

   Virtual memory is different from cache memory. Virtual memory allows the operating system to use secondary storage when RAM is insufficient. When required data is not present in physical memory, a **page fault** can occur, requiring data to be transferred from storage. This is much slower than cache access.

   Therefore, cloud servers should prioritize an efficient cache hierarchy and sufficient DRAM capacity. Technologies such as high-speed DDR memory can improve bandwidth, while SSD-based storage can reduce paging delays compared with traditional hard disks.

   Comparison

Feature	**Cache Hierarchy**	**Virtual Memory Paging**
Purpose	Fast CPU data access	Extends available memory
Speed	Very high	Much slower
Location	Near CPU	DRAM + secondary storage
Main problem	Cache miss	Page fault
Best improvement	Larger/better cache	More RAM + faster SSD

Recommendation

Use a **larger cache hierarchy, sufficient DRAM, and fast SSD storage**. The system should minimize paging because excessive paging significantly increases database-query latency.

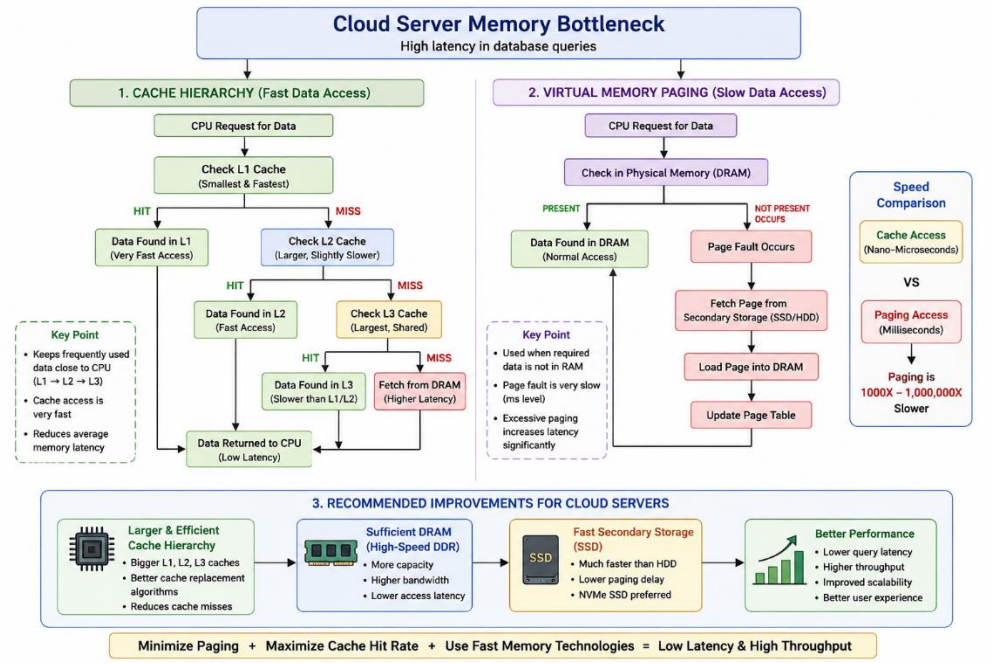
Output:

```
Online C Compiler

Enter database value to search: 50
Cache miss: DRAM access
Paging may add extra latency

Program executed successfully.
```

Diagram:



C Program

This program demonstrates a database search where the system first checks a small cache and then accesses the larger database when a cache miss occurs.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

Ans:

Explanation

An autonomous vehicle continuously receives large amounts of data from cameras, LiDAR, radar, GPS and other sensors. This data must be processed in real time. Therefore, the memory system must provide **high bandwidth and very low latency**.

SRAM is extremely fast and has low latency, making it suitable for small amounts of critical real-time data. However, SRAM is expensive and consumes more chip area, so it cannot normally be used for storing the entire sensor dataset.

DRAM provides much larger capacity at a lower cost. It is suitable for storing large sensor datasets and intermediate processing results, but its latency is higher than SRAM.

MRAM is a non-volatile memory technology. It can retain data without continuous power and can provide fast access with good endurance. It can be useful for storing important system states, configuration data and selected persistent information.

Proposed Memory Hierarchy

CPU/AI Accelerator → SRAM → DRAM → MRAM

- **SRAM:** Real-time sensor buffers and frequently accessed data
- **DRAM:** Large sensor datasets and processing buffers
- **MRAM:** Persistent system states and important data

This hierarchy minimizes latency by keeping time-critical information in SRAM while using DRAM for high-capacity storage and MRAM for non-volatile information.

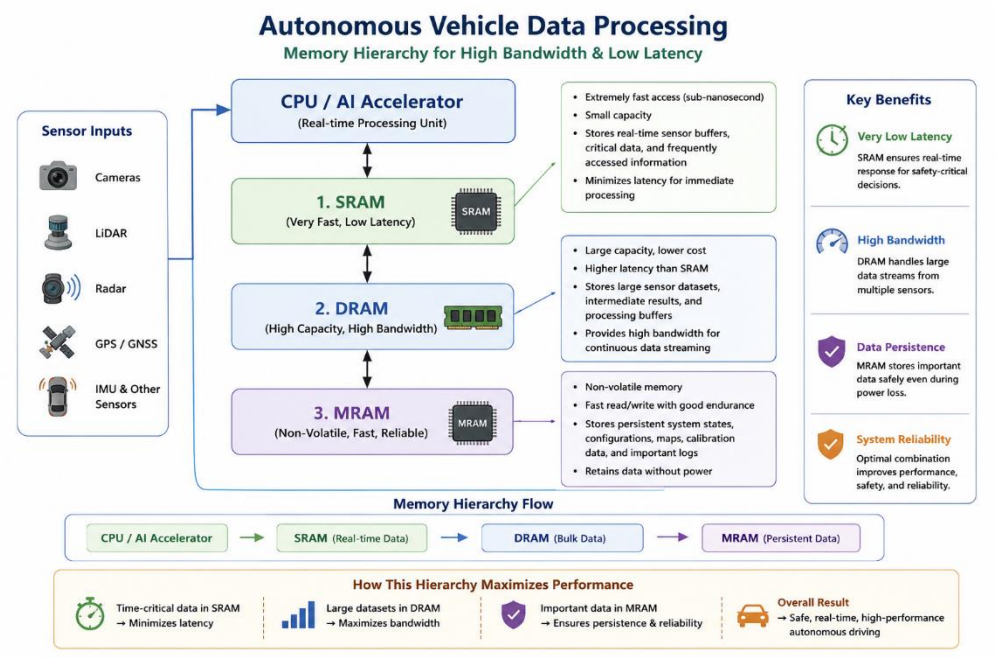
Output:

```
Online C Compiler

Autonomous Vehicle Memory Simulation
SRAM: ultra-fast real-time buffer
DRAM: large sensor-data storage
MRAM: non-volatile backup/state memory
Processed sensor values = 1024
Total = 130560

Program executed successfully.
```

Diagram:



C Program

The program demonstrates large sensor data being stored in DRAM, frequently required data being copied into a fast SRAM-like buffer, and MRAM being represented as non-volatile memory for system information.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

   Ans:

   Explanation

   An edge AI device must load and execute AI models quickly while consuming as little power as possible. The memory hierarchy must therefore balance **storage capacity, speed and energy consumption**.

   NAND Flash provides high-capacity non-volatile storage and can store the complete AI model even when the device is powered off. However, it is slower than DRAM for active computation.

   DRAM provides faster access and is therefore suitable for storing the AI model while it is being executed. However, DRAM consumes power and is volatile.

   RRAM, or Resistive RAM, is a non-volatile memory technology that can provide high density and potentially low-power operation. It is particularly promising for AI and edge-computing applications because model parameters can potentially be stored closer to the computation hardware.

   Proposed Hierarchy

AI Processor → RRAM → DRAM → NAND Flash

- **RRAM:** Frequently used AI parameters and fast-access model data
- **DRAM:** Active model and intermediate computations
- **NAND Flash:** Permanent storage of complete AI models

When an AI application starts, the required model can be transferred from NAND Flash to faster working memory. Frequently accessed parameters can then be placed in RRAM or other fast on-chip memory.

This reduces model-loading time and can lower energy consumption by reducing unnecessary transfers from slower storage.

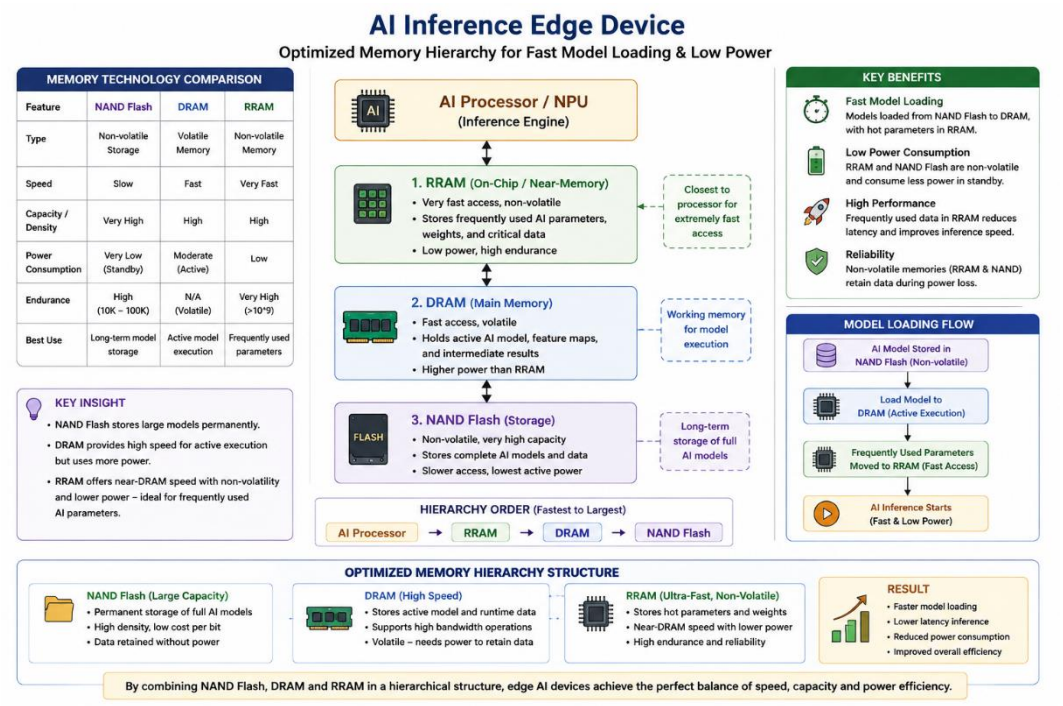
Output:

```
Online C Compiler

NAND Flash: permanent model storage
DRAM: active model memory
RRAM: fast and energy-efficient memory
Loaded model: 2 4 6 8 10 12
Model loading completed.

Program executed successfully.
```

Diagram:



C Program

The program represents NAND Flash as permanent model storage, DRAM as active working memory and RRAM as fast model-parameter storage.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

   Ans:

   Explanation

   Modern games contain large textures, 3D models, lighting information, maps and other scene data. During large scene loading, the processor and GPU need to access a huge amount of data. If the required data is repeatedly fetched from DRAM, memory latency and bandwidth limitations can cause **frame drops and reduced gaming performance**.

   L3 cache is much smaller than DRAM but provides significantly faster access. Frequently used game data, such as recently accessed textures, game objects and processing information, can be kept in the cache.

   DRAM provides much greater capacity and is therefore used for large game assets and scene data. However, repeated DRAM accesses can create a bottleneck.

   The solution is to improve the cache hierarchy and ensure that frequently accessed data remains in the L3 cache. Hardware prefetching and efficient data organization can also reduce unnecessary DRAM accesses.

   Proposed Hierarchy

   CPU/GPU → L1/L2 Cache → L3 Cache → High-Bandwidth DRAM

- **L1/L2:** Very frequently accessed data
- **L3:** Shared cache for frequently used game data
- **DRAM:** Large textures, maps and scene information

   Using high-bandwidth DRAM together with a larger and faster L3 cache can improve data transfer rates and reduce frame drops during complex scenes.

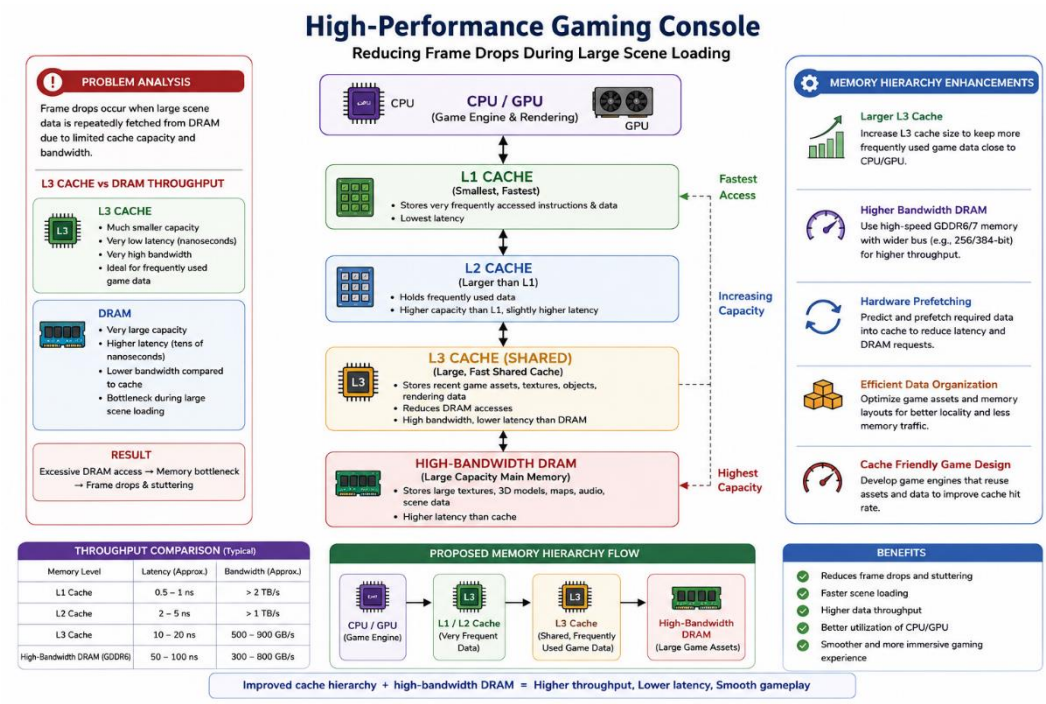
   Output:

```
Online C Compiler

Gaming Console Memory Simulation
L3 cache: fast access to hot scene data
DRAM: high-capacity scene storage
Frame data processed = 100000
Hot data sum = 202752
Using cache reduces repeated DRAM accesses.

Program executed successfully.
```

Diagram:



C Program

The program stores a large amount of scene data in DRAM and copies frequently accessed data into an L3-cache-like buffer. Repeated access to the smaller buffer represents the advantage of keeping frequently used data close to the processor.

***Code of Conduct**:*

I ___BHOOMIKA(192524291)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis